

OBJECT ORIENTED PROGRAMING

OOP

PRACTICALS

NAME: SYEDA FAIZA ASLAM

SECTION: B

ROLL NO: CT-25064

LAB NO: 05

COURSE CODE: CT-260

COURSE TEACHER: Mr. AHMED ZAKI

Exercise

1. Create a base class with the following members:

- Private integer privateInt

- Protected integer protectedInt

- Public integer publicInt

Create getters and setters for each of these variables. Derive 3 classes from the base class with the three types of inheritance based on visibility(public, protected, private). You can name these classes as publicChild, privateChild and protectedChild. After doing this, try and figure out which member you can access publicly or through getters and setters. Then print out the way that you were able to access them. For example, if you did private inheritance, you could not be able to access the members in the child classes, and would need to use their getters or setters.

SOURCECODE:

```
#include <iostream>
using namespace std;
```

```
class Base
{
private:
    int privateInt;

protected:
    int protectedInt;

public:
    int publicInt;
```

```

void setPrivateInt(int x) { privateInt = x; }
void setProtectedInt(int x) { protectedInt = x; }
void setPublicInt(int x) { publicInt = x; }

int getPrivateInt() { return privateInt; }
int getProtectedInt() { return protectedInt; }
int getPublicInt() { return publicInt; }
};

```

```

class publicChild : public Base
{
public:
    void show()
    {
        cout << getPrivateInt() << endl;
        cout << protectedInt << endl;
        cout << publicInt << endl;
    }
};

```

```

class protectedChild : protected Base
{
public:
    void setValues(int a, int b, int c)
    {
        setPrivateInt(a);
        setProtectedInt(b);
        setPublicInt(c);
    }

    void show()
    {
        cout << getPrivateInt() << endl;
        cout << protectedInt << endl;
        cout << publicInt << endl;
    }
};

```

```

    }
};

class privateChild : private Base
{
public:
    void setValues(int a, int b, int c)
    {
        setPrivateInt(a);
        setProtectedInt(b);
        setPublicInt(c);
    }

    void show()
    {
        cout << getPrivateInt() << endl;
        cout << protectedInt << endl;
        cout << publicInt << endl;
    }
};

int main()
{
    publicChild a;
    a.setPrivateInt(10);
    a.setProtectedInt(20);
    a.setPublicInt(30);
    a.show();

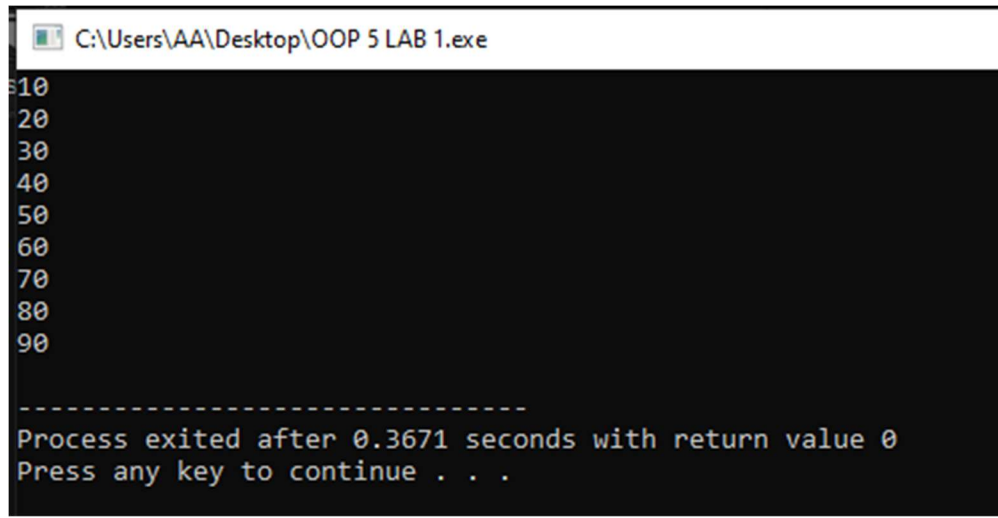
    protectedChild b;
    b.setValues(40, 50, 60);
    b.show();

    privateChild c;
    c.setValues(70, 80, 90);

```

```
c.show();  
  
return 0;  
}
```

OUTPUT:



```
C:\Users\AA\Desktop\OOP 5 LAB 1.exe  
10  
20  
30  
40  
50  
60  
70  
80  
90  
  
-----  
Process exited after 0.3671 seconds with return value 0  
Press any key to continue . . .
```

2. Create a class Teacher with the following private attributes: Name, Age, and Institute. Derive three classes from it that has the following names: HumanitiesTeacher, ScienceTeacher, MathsTeacher. These classes should have the following members:

- Department (this should have the value “science”, “maths” or “humanities”)
- Course Name
- Designation (for example, lecturer, professor, etc)

Create proper accessors and mutators for the attributes. Create objects for each of the classes and display the values. You can ask the user to input the values.

SOURCECODE:

```
#include <iostream>
#include <string>
using namespace std;
```

```
class Teacher
{
private:
    string name;
    int age;
    string institute;

public:
    void setName(string n) { name = n; }
    void setAge(int a) { age = a; }
    void setInstitute(string i) { institute = i; }

    string getName() { return name; }
    int getAge() { return age; }
    string getInstitute() { return institute; }
};
```

```
class HumanitiesTeacher : public Teacher
{
private:
    string courseName;
    string designation;

public:
    void setCourseName(string c) { courseName = c; }
    void setDesignation(string d) { designation = d; }

    void display()
    {
```

```

        cout << "\n----- Humanities Teacher Details ----- \n";
        cout << "Name: " << getName() << endl;
        cout << "Age: " << getAge() << endl;
        cout << "Institute: " << getInstitute() << endl;
        cout << "Department: Humanities" << endl;
        cout << "Course Name: " << courseName << endl;
        cout << "Designation: " << designation << endl;
    }
};

```

```

class ScienceTeacher : public Teacher
{
private:
    string courseName;
    string designation;

public:
    void setCourseName(string c) { courseName = c; }
    void setDesignation(string d) { designation = d; }

    void display()
    {
        cout << "\n----- Science Teacher Details ----- \n";
        cout << "Name: " << getName() << endl;
        cout << "Age: " << getAge() << endl;
        cout << "Institute: " << getInstitute() << endl;
        cout << "Department: Science" << endl;
        cout << "Course Name: " << courseName << endl;
        cout << "Designation: " << designation << endl;
    }
};

```

```

class MathsTeacher : public Teacher
{
private:

```

```
string courseName;  
string designation;
```

```
public:
```

```
void setCourseName(string c) { courseName = c; }  
void setDesignation(string d) { designation = d; }
```

```
void display()  
{  
    cout << "\n----- Maths Teacher Details ----- \n";  
    cout << "Name: " << getName() << endl;  
    cout << "Age: " << getAge() << endl;  
    cout << "Institute: " << getInstitute() << endl;  
    cout << "Department: Maths" << endl;  
    cout << "Course Name: " << courseName << endl;  
    cout << "Designation: " << designation << endl;  
}  
};
```

```
int main()
```

```
{  
    int choice;  
  
    cout << "\n----- Teacher Department Menu ----- \n";  
    cout << "1. Humanities Teacher \n";  
    cout << "2. Science Teacher \n";  
    cout << "3. Maths Teacher \n";  
    cout << "Enter Choice: ";  
    cin >> choice;  
    cin.ignore();
```

```
    string name, institute, course, designation;  
    int age;
```

```
    cout << "\nEnter Name: ";
```



```
getline(cin, name);
```

```
cout << "Enter Age: ";
```

```
cin >> age;
```

```
cin.ignore();
```

```
cout << "Enter Institute: ";
```

```
getline(cin, institute);
```

```
cout << "Enter Course Name: ";
```

```
getline(cin, course);
```

```
cout << "Enter Designation: ";
```

```
getline(cin, designation);
```

```
if(choice == 1)
```

```
{
```

```
    HumanitiesTeacher h;
```

```
    h.setName(name);
```

```
    h.setAge(age);
```

```
    h.setInstitute(institute);
```

```
    h.setCourseName(course);
```

```
    h.setDesignation(designation);
```

```
    h.display();
```

```
}
```

```
else if(choice == 2)
```

```
{
```

```
    ScienceTeacher s;
```

```
    s.setName(name);
```

```
    s.setAge(age);
```

```
    s.setInstitute(institute);
```

```
    s.setCourseName(course);
```

```
    s.setDesignation(designation);
```

```
    s.display();
```

```
}
```

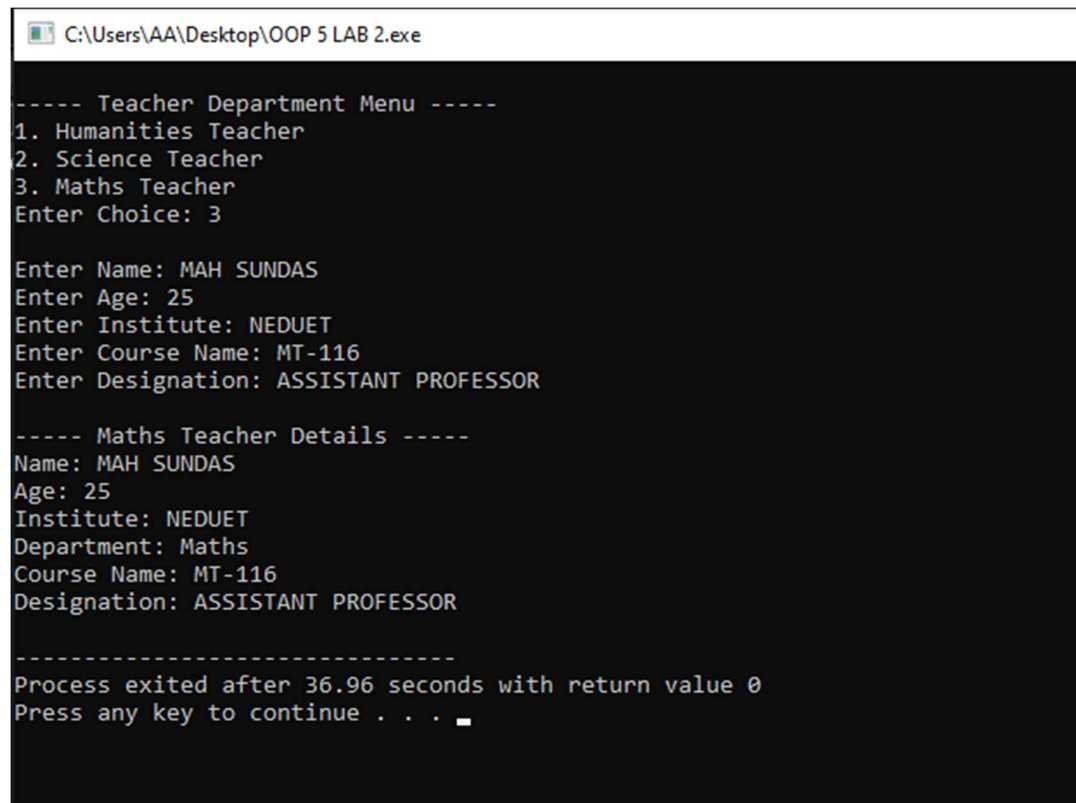
```

else if(choice == 3)
{
    MathsTeacher m;
    m.setName(name);
    m.setAge(age);
    m.setInstitute(institute);
    m.setCourseName(course);
    m.setDesignation(designation);
    m.display();
}
else
{
    cout << "Invalid Choice!" << endl;
}

return 0;
}

```

OUTPUT:



```

C:\Users\AA\Desktop\OOP 5 LAB 2.exe

----- Teacher Department Menu -----
1. Humanities Teacher
2. Science Teacher
3. Maths Teacher
Enter Choice: 3

Enter Name: MAH SUNDAS
Enter Age: 25
Enter Institute: NEDUET
Enter Course Name: MT-116
Enter Designation: ASSISTANT PROFESSOR

----- Maths Teacher Details -----
Name: MAH SUNDAS
Age: 25
Institute: NEDUET
Department: Maths
Course Name: MT-116
Designation: ASSISTANT PROFESSOR

-----
Process exited after 36.96 seconds with return value 0
Press any key to continue . . .

```

3. A defense organization is making a hierarchy of different types of weapons. They have classified the nuclear bomb as follows: Weapons → Hot Weapons → Bombs → Nuclear Bombs. Create classes and apply inheritance as necessary for the above hierarchy.

Each class should have a method called: “xxxxxDescription”, where xxxx would be class name. The method should print out what that weapon does. Eg. Hot weapons uses gunpowder, or explode. Bombs blow up. Nuclear bombs blow up, and use nuclear fission and fusion.

SOURCECODE:

```
#include <iostream>
using namespace std;
```

```
class Weapons
{
public:
    void WeaponsDescription()
    {
        cout << "Weapons are used for attack." << endl;
    }
};
```

```
class HotWeapons : public Weapons
{
public:
    void HotWeaponsDescription()
    {
        cout << "Hot weapons use gunpowder." << endl;
    }
};
```

```
class Bombs : public HotWeapons
```

```

{
public:
    void BombsDescription()
    {
        cout << "Bombs blow up." << endl;
    }
};

class NuclearBombs : public Bombs
{
public:
    void NuclearBombsDescription()
    {
        cout << "Nuclear bombs use fission and fusion." << endl;
    }
};

int main()
{
    NuclearBombs n;

    n.WeaponsDescription();
    n.HotWeaponsDescription();
    n.BombsDescription();
    n.NuclearBombsDescription();

    return 0;
}

```

OUTPUT:

```
C:\Users\AA\Desktop\OOP 5 LAB 3.exe
Weapons are used for attack.
Hot weapons use gunpowder.
Bombs blow up.
Nuclear bombs use fission and fusion.

-----
Process exited after 1.048 seconds with return value 0
Press any key to continue . . .
```

4. A bakery is making a program to calculate the bills for each of their customers. The items in the bakery are categorized as follows:

Item:

- Name

- Quantity

Baked Goods: (derived from item)

- Discount = 10%

Cakes: (derived from baked goods)

- Price = 600

Bread: (derived from baked goods)

- Price = 200

Drinks: (derived from item)

- Discount = 5%

- Price = 100

In your main function, as a bakery personnel, input the items and their quantities and calculate the bill accordingly.

SOURCECODE:

```
#include <iostream>
#include <string>
using namespace std;
```

```
class Item
{
protected:
    string name;
    int quantity;

public:
    void setItem(string n, int q)
    {
        name = n;
        quantity = q;
    }
};
```

```
class BakedGoods : public Item
{
protected:
    float discount;

public:
    BakedGoods()
    {
        discount = 0.10;
    }
};
```

```
class Cakes : public BakedGoods
{
private:
```

```
float price;
```

```
public:
```

```
    Cakes()
```

```
    {
```

```
        price = 600;
```

```
    }
```

```
float bill()
```

```
{
```

```
    float total = price * quantity;
```

```
    return total - (total * discount);
```

```
}
```

```
void display()
```

```
{
```

```
    cout << "\n----- Cake Bill ----- \n";
```

```
    cout << "Item Name: " << name << endl;
```

```
    cout << "Quantity: " << quantity << endl;
```

```
    cout << "Price per Item: " << price << endl;
```

```
    cout << "Discount: 10%" << endl;
```

```
    cout << "Total Bill: " << bill() << endl;
```

```
}
```

```
};
```

```
class Bread : public BakedGoods
```

```
{
```

```
private:
```

```
    float price;
```

```
public:
```

```
    Bread()
```

```
    {
```

```
        price = 200;
```

```
    }
```

```
float bill()
{
    float total = price * quantity;
    return total - (total * discount);
}
```

```
void display()
{
    cout << "\n----- Bread Bill ----- \n";
    cout << "Item Name: " << name << endl;
    cout << "Quantity: " << quantity << endl;
    cout << "Price per Item: " << price << endl;
    cout << "Discount: 10%" << endl;
    cout << "Total Bill: " << bill() << endl;
}
};
```

```
class Drinks : public Item
{
private:
    float discount;
    float price;
```

```
public:
    Drinks()
    {
        discount = 0.05;
        price = 100;
    }
```

```
float bill()
{
    float total = price * quantity;
    return total - (total * discount);
}
```



```

    }

    void display()
    {
        cout << "\n----- Drink Bill ----- \n";
        cout << "Item Name: " << name << endl;
        cout << "Quantity: " << quantity << endl;
        cout << "Price per Item: " << price << endl;
        cout << "Discount: 5%" << endl;
        cout << "Total Bill: " << bill() << endl;
    }
};

int main()
{
    int choice, qty;

    cout << "\n----- Bakery Menu ----- \n";
    cout << "1. Cake (Price: 600, Discount: 10%) \n";
    cout << "2. Bread (Price: 200, Discount: 10%) \n";
    cout << "3. Drink (Price: 100, Discount: 5%) \n";
    cout << "Enter Choice: ";
    cin >> choice;

    cout << "Enter Quantity: ";
    cin >> qty;

    if(choice == 1)
    {
        Cakes c;
        c.setItem("Cake", qty);
        c.display();
    }
    else if(choice == 2)
    {

```

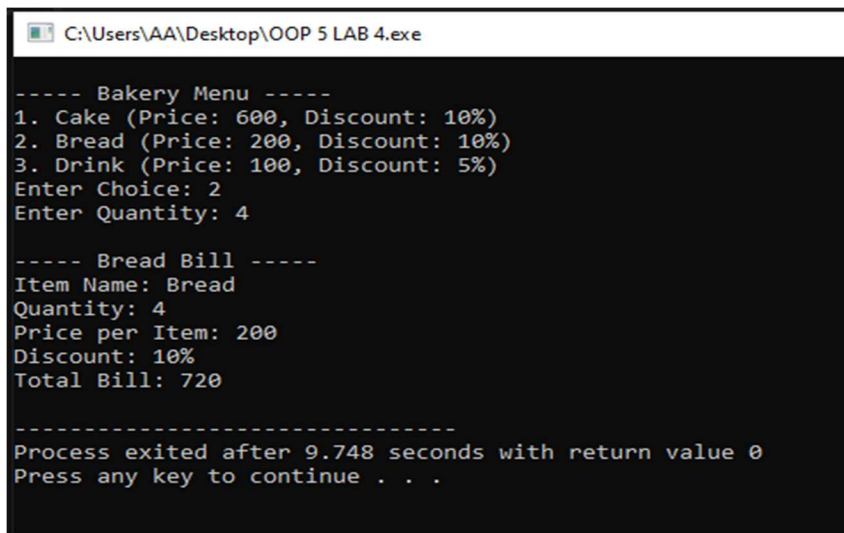
```

        Bread b;
        b.setItem("Bread", qty);
        b.display();
    }
    else if(choice == 3)
    {
        Drinks d;
        d.setItem("Drink", qty);
        d.display();
    }
    else
    {
        cout << "Invalid Choice!" << endl;
    }

    return 0;
}

```

OUTPUT:



```

C:\Users\AA\Desktop\OOP 5 LAB 4.exe
----- Bakery Menu -----
1. Cake (Price: 600, Discount: 10%)
2. Bread (Price: 200, Discount: 10%)
3. Drink (Price: 100, Discount: 5%)
Enter Choice: 2
Enter Quantity: 4

----- Bread Bill -----
Item Name: Bread
Quantity: 4
Price per Item: 200
Discount: 10%
Total Bill: 720

-----
Process exited after 9.748 seconds with return value 0
Press any key to continue . . .

```

SOLVED EXAMPLES:

1) Single Inheritance Code

SOURCECODE:

```
#include <iostream>
using namespace std;

class base
{
public:
    int x;

    void getdata()
    {
        cout << "Enter the value of x = ";
        cin >> x;
    }
};

class derive : public base
{
private:
    int y;

public:
    void readdata()
    {
        cout << "Enter the value of y = ";
        cin >> y;
    }

    void product()
    {
```

```

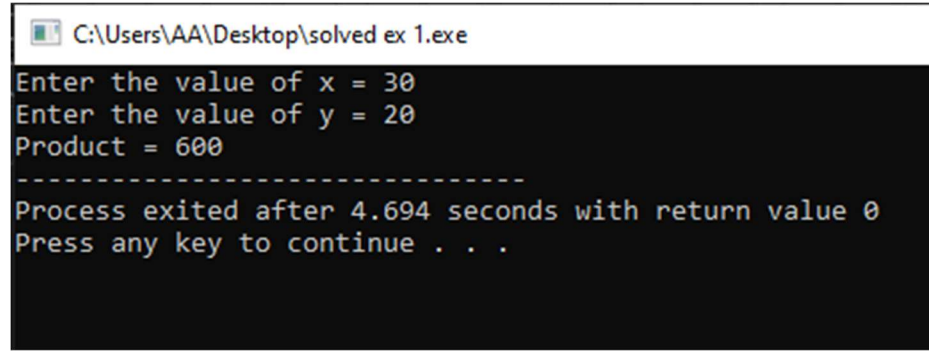
        cout << "Product = " << x * y;
    }
};

int main()
{
    derive a;
    a.getdata();
    a.readdata();
    a.product();

    return 0;
}

```

OUTPUT:



```

C:\Users\AA\Desktop\solved ex 1.exe
Enter the value of x = 30
Enter the value of y = 20
Product = 600
-----
Process exited after 4.694 seconds with return value 0
Press any key to continue . . .

```

2) Multilevel Inheritance Code

SOURCECODE:

```

#include <iostream>
using namespace std;

class base
{
public:
    int x;

    void getdata()
    {

```

```

        cout << "Enter value of x = ";
        cin >> x;
    }
};

class derive1 : public base
{
public:
    int y;

    void readdata()
    {
        cout << "Enter value of y = ";
        cin >> y;
    }
};

class derive2 : public derive1
{
private:
    int z;

public:
    void indata()
    {
        cout << "Enter value of z = ";
        cin >> z;
    }

    void product()
    {
        cout << "Product = " << x * y * z;
    }
};

int main()
{

```

```

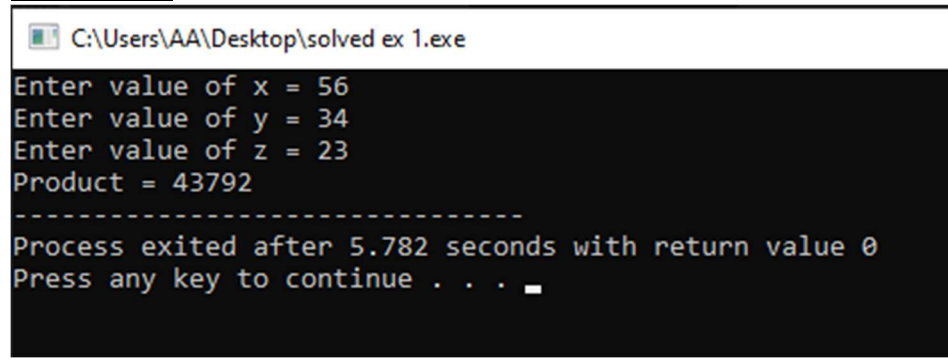
    derive2 a;

    a.getdata();
    a.readdata();
    a.indata();
    a.product();

    return 0;
}

```

OUTPUT:



```

C:\Users\AA\Desktop\solved ex 1.exe
Enter value of x = 56
Enter value of y = 34
Enter value of z = 23
Product = 43792
-----
Process exited after 5.782 seconds with return value 0
Press any key to continue . . . _

```

3) Hierarchical Inheritance Code:

SOURCECODE:

```

#include <iostream>
using namespace std;

class A
{
public:
    int x, y;

    void getdata()
    {
        cout << "Enter value of x and y: ";
        cin >> x >> y;
    }
};

```

```
class B : public A
{
public:
    void product()
    {
        cout << "Product = " << x * y << endl;
    }
};
```

```
class C : public A
{
public:
    void sum()
    {
        cout << "Sum = " << x + y << endl;
    }
};
```

```
int main()
{
    B obj1;
    C obj2;

    obj1.getdata();
    obj1.product();

    obj2.getdata();
    obj2.sum();

    return 0;
}
```

OUTPUT:

```
C:\Users\AA\Desktop\solved ex 1.exe
Enter value of x and y: 30
40
Product = 1200
Enter value of x and y: 2
3
Sum = 5

-----
Process exited after 15.21 seconds with return value 0
Press any key to continue . . .
```